— USERNAME AVAILABILITY

How apps say "taken" in milliseconds

00 The problem

Billions of usernames. A database query on every keystroke is too slow and too expensive at scale.

01 Bloom filter (in memory)

- A bit array of **m** bits + **k** hash functions.
- **Add a name:** hash it k times, set those k bits to 1.
- **Check a name:** hash it, look at the same k bits.

Any bit is 0
Definitely free. No DB call.

All bits are 1
Maybe taken. Check further.

No false negatives. False positives are possible.

02 Cache

Only for "maybe taken". Redis holds recently checked and popular names.

03 Database

Indexed lookup on `username` gives the real answer.

04 Final call at signup

A `UNIQUE(username)` constraint. If two people grab the same name at once, the database lets only one win.

— INTERVIEW READY

Numbers and trade-offs to quote

01 Sizing rule of thumb

- About **10 bits per item** with ~7 hashes gives **~1%** false positives.
- 1 billion names at 1% ≈ **1.2 GB** of memory.
- Best hash count: `k = (m / n) × ln 2`

02 Trade-offs

- **No deletes** in a standard Bloom filter. Use a counting Bloom filter if names get freed.
- False positive rate rises as it fills. Size for growth or rebuild it.
- Only saves work on the “free” path. “Maybe” still costs a lookup.

03 Used in real systems

Cassandra, HBase and RocksDB use Bloom filters to skip disk reads for keys that aren't there.

04 Say it in 20 seconds

“Check a Bloom filter first. If any bit is 0, the name is free. If all are 1, check the cache, then the indexed DB. A unique constraint at signup handles races.”

05 Common mistakes

- Saying a Bloom filter **confirms** a name exists. It can't.
- Forgetting the unique constraint at signup.